INHA UNIVERSITY | INTEGRATED SYSTEM ENGINEERING

SMART MOBILITY HUB

ROS 2 Robotics Student Implementation Handbook

From a 35-page assignment to an engineering implementation roadmap

Contents

1 Mission, boundaries, and learning roadmap **1**

1.1 The transport-hub challenge . . . 1
1.2 Nonnegotiable design decisions . . . 1
1.3 System concept diagram . . . 1
1.4 How to use this handbook . . . 2

2 Ubuntu, ROS 2, and reproducible workspace **3**

2.1 Version policy . . . 3
2.2 Package creation . . . 3
2.3 Repo and run policy . . . 3
2.4 Simulator time . . . 4

3 Map, graph, ROS interfaces, and contracts **5**

3.1 Canonical map M1 . . . 5
3.2 Motion targets and safety envelope . . . 5
3.3 Topic dictionary . . . 5
3.4 Required service and action names . . . 6
3.5 Interfaces exercise . . . 6

4 Scenario-by-scenario implementation playbook **7**

4.1 S1 — Passenger counting and crossing occupancy . . . 7
4.2 S2 — Thirty-second passenger-demand prediction . . . 7
4.3 S3 — Cancellable bus docking and boarding . . . 8
4.4 S4 — Idempotent parking reservation and barrier control . . . 8
4.5 S5 — Congestion and obstruction estimation . . . 8
4.6 S6 — Optimal graph routing and safe replanning . . . 9
4.7 S7 — Robot escort with human readiness and cancellation . . . 9
4.8 S8 — Concurrent hub orchestration . . . 10
4.9 S9 — Deliberate fault, emergency stop, and recovery . . . 10

5 Implementation architecture, concurrency and safety **11**

5.1 Node ownership and communication . . . 11
5.2 Crossing ownership sequence . . . 11
5.3 Control-loop pseudocode . . . 11
5.4 Timer and callback policy . . . 12
5.5 TF2 policy and frames . . . 12

6 Testing, reproducibility, and evidence **13**

6.1 Build–launch–inspect–record cycle . . . 13
6.2 Mandatory verification matrix . . . 13
6.3 Metrics and statistics . . . 14
6.4 Standalone reference tests actually run during preparation . . . 14

7 Seven-week student plan and graded outputs **15**
7.1 Team-of-five responsibility map . . . 15
7.2 Assessment — source rubric unchanged . . . 15
7.3 Eight-minute demonstration storyboard . . . 16

8 Prioritized optional engineering improvements **17**
8.1 Practical extension design exercise . . . 17

9 Coverage checklist and quality-verification report **18**
9.1 Requirement-to-implementation traceability . . . 18
9.2 Submission readiness checks . . . 18
9.3 Verification performed for this guide . . . 18

A Companion code: pure Python route planner **20**

B Companion code: parking domain logic **21**

C Companion code: local unit tests **23**

D Companion ROS 2 teaching examples **24**

CHAPTER 1

Mission, boundaries, and learning roadmap

Assignment fidelity — The source is the 35-page *Smart Mobility Hub* assignment for Smart Mobility Engineering Lab, Inha University. It specifies eight functional scenarios S1–S8 and a ninth, mandatory safety/fault suite S9; T01–T18 are required. This guide preserves those conditions rather than substituting a generic mobile robot tutorial.

1.1 The transport-hub challenge

Build a simulated 12 m × 8 m Songdo station. Passengers enter a queue; a camera estimates occupancy; an AI predicts demand; a bus docks and boards passengers; cars reserve parking; congestion changes routes; an assistant robot escorts a person; and a safety authority prevents incompatible crossing movements. Every decision must have a timestamped, replayable consequence. The model is *educational*, not a deployable autonomous road system.

1.2 Nonnegotiable design decisions

1. **Mandatory simulator:** deterministic Python 2D physics and RViz markers; Gazebo and Nav2 are optional, not baseline prerequisites.
2. **Communication:** topics represent streams, services handle bounded transactions, actions handle long-running cancellable missions; all application nodes use `rclpy`.
3. **Command authority:** only `safety_supervisor` writes final velocities; managers write candidate commands; `actuator_bridge` independently expires final commands.
4. **Physical scale:** optional 3 m × 2 m tabletop uses a 1:4 adapter. Do not change logical map coordinates.
5. **Reproducibility:** seed, scenario, configuration hash, model ID, experiment run ID and code commit identify each result.

1.3 System concept diagram

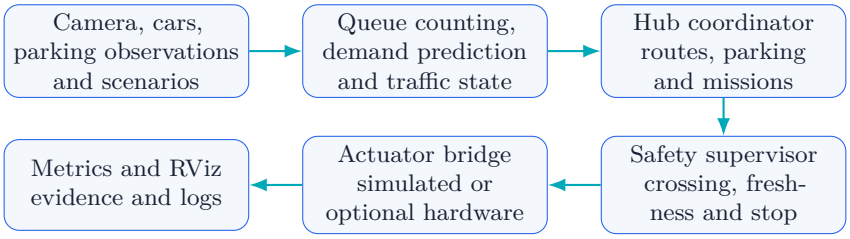


Figure 1.1: Assessed data flow. The only path from mission commands to movement passes through the safety layer.

1.4 How to use this handbook

Work sequentially through environment setup, contracts, pure algorithms, ROS wrappers, integration and tests. **Do not** interpret a short code example as a completed node unless its dependencies and acceptance tests are present. The `code/` folder contains runnable pure-Python reference modules and isolated ROS teaching examples; complete integrated node implementations remain student development tasks.

CHAPTER 2

Ubuntu, ROS 2, and reproducible workspace

2.1 Version policy

Use Ubuntu 24.04 LTS with ROS 2 Jazzy on a native installation or compatible Ubuntu environment. The assignment also permits Humble; do not mix distributions among teammates. This handbook targets Jazzy. **A ROS runtime is not available in this document-generation environment; commands below are prescribed, not execution-verified.**

Installation checklist

1. Confirm OS: `lsb_release -a`; inspect `uname -m`.
2. Follow the official Jazzy Ubuntu `deb` installation procedure, including ROS repository and GPG key setup. Do not paste unaudited package-source commands into privileged shells.
3. Install the ROS desktop distribution, development tools, and package dependencies.
4. Use `rosdep` for ROS dependencies and a virtual environment only for compatible non-ROS Python libraries; avoid replacing ROS's system Python installation.

```
source /opt/ros/jazzy/setup.bash
sudo apt update
sudo apt install -y python3-colcon-common-extensions python3-rosdep \\
  python3-pytest python3-numpy python3-sklearn python3-opencv \\
  ros-jazzy-rviz2 ros-jazzy-ros2bag ros-jazzy-rosbag2-storage-default-plugins
mkdir -p ~/smart_hub_ws/src
cd ~/smart_hub_ws
ros2 doctor --report
```

Checkpoint: `ros2 -help`, `ros2 pkg list`, and `ros2 doctor` return usable output. Save command transcripts with date and machine information. If `rosdep` is uninitialized, follow its displayed initialization instruction once per machine.

2.2 Package creation

```
cd ~/smart_hub_ws/src
ros2 pkg create smart_hub_interfaces --build-type ament_cmake
ros2 pkg create smart_hub --build-type ament_python \\
  --dependencies rclpy std_msgs sensor_msgs geometry_msgs \\
  nav_msgs visualization_msgs smart_hub_interfaces
cd ~/smart_hub_ws
rosdep install --from-paths src --ignore-src -r -y
colcon build --symlink-install
source install/setup.bash
```

The interface package requires `rosidl_default_generators` and `rosidl_default_runtime`; add each `msg/`, `srv/`, and `action/` file to `rosidl_generate_interfaces`, declare its message dependencies and export runtime dependencies. Add `<member_of_group>rosidl_interface_packages</member_of_group>` in `package.xml`. After a clean build, inspect interfaces with `ros2 interface show`.

2.3 Repo and run policy

```
cd ~/smart_hub_ws
git init
git add src
git commit -m 'Freeze hub interface and map contracts'
export ROS_DOMAIN_ID=27  # choose a unique team value, 0..101
ros2 node list
```

Never commit recorded personal video without permission; distribute annotations or a lawful source link. Explicitly select simulation or hardware via a launch argument. Hardware starts with propulsion disabled.

2.4 Simulator time

The baseline can run on ROS wall time using a fixed 0.05-second step and deterministic seeded event schedule; measuring watchdog age uses a monotonic local clock. If a future simulator publishes `/clock`, set `use_sim_time` consistently, publish only one clock, and continue hardware watchdogs when simulated time pauses.

CHAPTER 3

Map, graph, ROS interfaces, and contracts

3.1 Canonical map M1

Origin southwest; $+x$ east, $+y$ north. Positions are metres, yaw is radians. Crossing C is $[4.5, 5.5] \times [1.25, 4.75]$; queue Q is $[2, 6] \times [5, 7]$. Bus bay B is $(8, 4)$; assistance robot starts R at $(3, 6)$; destination D is $(9, 1)$; P1–P3 lie at $(8, 6.5)$, $(9.5, 6.5)$, $(11, 6.5)$. Obstacle O is centered at $(7, 2)$ with radius 0.35 m.

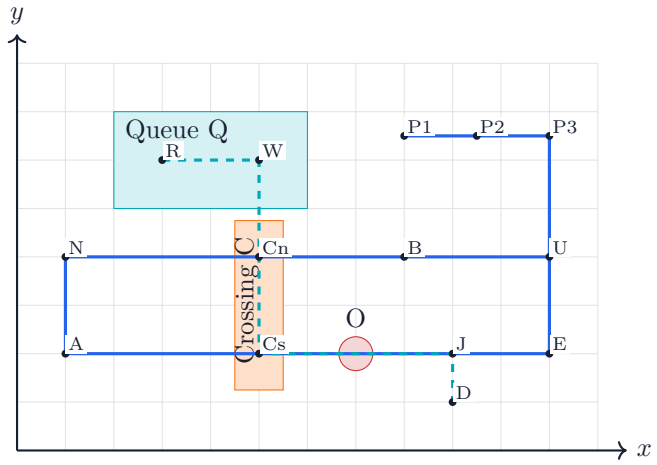


Figure 3.1: Educational M1 map and graph. Crossing ownership is determined by swept footprints, not by visual line intersections alone.

3.2 Motion targets and safety envelope

Target	Baseline acceptance value
Robot/bus cap	0.25/0.40 m/s
Robot/vehicle radius	0.20/0.30 m
Extra clearance	0.20 m
Candidate command watchdog	0.50 s
Perception freshness	1.0 s
Stop receipt to zero command	$\leq$ 0.20 s, simulation
Arrival stability	0.20 m, 0.17 rad, hold 1 s
Crossing clear debounce	2.0 s

Collision checks must use swept footprints. Segment–rectangle intersection is needed for crossing-related edges including N–Cn, Cn–B, A–Cs, Cs–J, and Cn–Cs. Do not grant a new owner until the entire previous footprint is observed clear.

3.3 Topic dictionary

Topic	Message type	Nominal rate
/camera/image_raw	sensor_msgs/Image	10 Hz
/hub/queue	QueueState	5 Hz
/hub/demand	DemandPrediction	1 Hz
/hub/vehicles	VehicleStateArray	10 Hz
/hub/bus_arrival	BusArrival	2 Hz
/parking/observed	SlotObservationArray	5 Hz
/parking/state	ParkingState	2 Hz + change
/traffic/edges	EdgeStateArray	2 Hz
/crossing/state	CrossingState	10 Hz
/robot/odom	nav_msgs/Odometry	20 Hz
/robot/cmd_vel_candidate	TwistStamped	20 Hz
/robot/cmd_vel	Twist	20 Hz
/bus/cmd_vel_candidate	TwistStamped	20 Hz
/bus/cmd_vel	Twist	20 Hz
/hub/estop	Bool	event + 10 Hz
/hub/safety	SafetyState	10 Hz
/hub/events	HubEvent	on change
/hub/markers	MarkerArray	5 Hz

All custom types are in `smart_hub_interfaces`. Use sensor-data QoS for camera reception, reliable R10 for queue/demand/edge state, reliable R1 transient-local for parking/crossing/safety state, reliable R1 commands and reliable R100 events. Confirm publisher/subscriber QoS compatibility rather than assuming the topic name suffices.

3.4 Required service and action names

Name	Interface and purpose
/parking/reserve_slot	`ReserveParking`: atomic, idempotent assignment
/parking/release_slot	`ReleaseParking`: token-validated release
/route/compute	`ComputeRoute`: short Dijkstra transaction
/crossing/request	`RequestCrossing`: ACQUIRE/RENEW/RELEASE
/scenario/reset	`ResetScenario`: seeded reset and new run ID
/safety/clear_stop	`ClearSafetyStop`: deliberate reset after proof
/robot/confirm_ready	`std_srvs/Trigger`: readiness at pickup
/bus/dock	`DockBus.action`: cancellable docking
/robot/escort	`EscortPassenger.action`: multi-phase escort
/robot/navigate_to	`NavigateTo.action`: reusable navigation

See the original assignment sections 6–7 for *exact* custom field definitions. Do not change their fields to match a code example. Headers identify observation time; preserve original timestamps during republishing. All top-level state messages carry headers; a stale zero is not a valid empty queue.

3.5 Interfaces exercise

Run `ros2 interface show smart_hub_interfaces/srv/ComputeRoute`; provide two sample requests and demonstrate a failure response `NO_ROUTE`. Inspect publish rate with `ros2 topic hz /hub/queue` and transport configuration with `ros2 topic info /hub/queue -verbose`.

CHAPTER 4

Scenario-by-scenario implementation playbook

Development contract — Each section supplies an algorithm, wiring plan, testing sequence, and expected evidence. The full assignment's message schemas and exact service/action signatures remain authoritative. The accompanying code files implement selected pure algorithms and isolated ROS examples, not a fully integrated submitted solution.

4.1 S1 — Passenger counting and crossing occupancy

Owner M1. Subscribe to `/camera/image_raw`; a synthetic-track path is also required for deterministic tests. Calibrate queue polygon Q and crossing C independently. In video mode, detect people, associate bounding boxes by persistent track ID and project their bottom-centres to map ground points; with four or more planar calibration correspondences use a homography. Keep a 0.4-second track confirmation debounce and tolerate 0.5-second short occlusion. Publish `QueueState` at 5 Hz with accurate observation timestamps. If the image feed freezes, publish `valid=false` after one second, never relabel old frames as newly observed. Maintain crossing occupied until two seconds of continuous clear observations.

S1 build order

1. Generate fixed track events at t=5,10,15,20,24,30 s. 2. Implement inside-polygon, track persistence and observation-age tests as pure functions. 3. Subscribe to synthetic tracks and an actual prerecorded camera/video feed; document detector version and confidence threshold. 4. Publish queue counts, confidence and crossing occupied flag. 5. Render overlays and log every change with a run ID.

Pass: counts 0/3/5/4 after debounce, no duplicate identity during 0.3 s occlusion, invalid within 1.2 s of freeze, and no incompatible crossing entry. Annotate ≥ 100 video frames; target MAE ≤ 1 person and report detection precision/recall if labels exist. **Evidence:** annotated frames, count-versus-truth plot, ROS topic capture and crossing-stop video.

4.2 S2 — Thirty-second passenger-demand prediction

Owner M1, dispatch owner M2. Consume `/hub/queue` at 5 Hz; prepare one-second history over 60 seconds. Train a documented ridge regressor on queue count, rolling mean, ten-second trend, and thirty-second arrival/departure totals; distinguish real track-derived arrivals from synthetic events. Create at least 30 independent six-minute episodes and split episodes 60/20/20, not overlapping windows, before fitting the model. No future arrival plan or hidden state is allowed as an online feature. Persistence $\hat{q}_{t+30} = q_t$ is mandatory as a baseline.

S2 development sequence

1. Save episode seeds and separate split IDs. 2. Fit preprocessing on training episodes only and choose hyperparameters on validation episodes. 3. Hold predictions invalid until features are complete or during stale input. 4. Publish predicted and baseline values at 1 Hz with a model ID. 5. Have the coordinator request `/bus/dock` only after forecast ≥ 6 for three valid samples, with one outstanding goal and a 30-second cooldown. 6. Reset escalation after five forecasts below 4; use a labeled observed-count fallback when model input is invalid.

Pass: held-out MAE no worse than persistence, p95 inference under 100 ms, exactly one dispatch per outstanding bus mission, no invalid-forecast dispatch. Measure downstream passenger waiting time under predictive versus

reactive policy using identical episode seeds. Include at least one documented case where the model performs worse.

4.3 S3 — Cancellable bus docking and boarding

Owner M2. `bus_manager` subscribes to `/hub/vehicles` and `/hub/safety`, publishes arrival state and a candidate `TwistStamped`; it serves `DockBus` and asynchronously calls route and crossing services. Use `/bus/door_request` for commands and `/bus/door_open` for measured door status. States: `IDLE, APPROACH, WAIT, DOCK, BOARD, COMPLETE, SAFE_STOP`. Plan A–N–Cn–B by default, stop outside crossing until granted, renew token while crossing, release only after full clearance. Dock after stable pose within 0.20 m and 0.17 rad for one second and speed below 0.02 m/s. Board up to four people over eight seconds with zero propulsion; close door and confirm closure before completion or any departure.

S3 action implementation

1. Add `DockBus.action` from original section 7.2. 2. Register goal/cancel/execute callbacks using a multithreaded executor or short state-machine ticks. 3. Reject busy, unsafe, invalid and unreachable requests. 4. Publish feedback every 0.5 s and calculate ETA using remaining graph length; mark unavailable ETA invalid. 5. Implement safe cancellation: command zero, confirm stop, then report canceled. 6. Add measured door interlocks to supervisor and actuator bridge.

Pass: ten of ten nominal dockings, position error ≤ 0.20 m, door never open while moving, command zero within 0.5 s after cancellation acceptance; measure ETA error at five metres remaining. Save complete action trace, crossing token trace, door acknowledgment and measured speed.

4.4 S4 — Idempotent parking reservation and barrier control

Owner M3. Three slots begin UNKNOWN, transition after 0.5 s of stable valid observations, and may take FREE, RESERVED, OCCUPIED or UNKNOWN. Only freshly sensed FREE slots are reservable. Use a mutex/serialized callback group covering validation, request-ID cache, preferred-slot selection, token creation, occupancy ownership and expiry; never reserve the same slot twice. TTL range 5–120 s. Duplicate request ID plus identical payload returns the original result; reused ID with different payload is rejected. On timeout release only administrative ownership: a physically occupied or stale slot cannot become FREE. Opening the barrier additionally requires fresh clear entrance sensing and an authorized owner; compare request with independent status `/parking/barrier_open`.

S4 code practice

Study `code/parking_engine.py`; its tested atomicity/idempotency logic is an isolated domain example. Add ROS services `/parking/reserve_slot` and `/parking/release_slot` wrapping that domain logic, a time-stamped debounced observation adapter, ROS ParkingState publisher and an asynchronous barrier acknowledgment handler. Keep physical occupancy and administrative reservation distinct.

Pass: begin P1 occupied, P2/P3 free; C1/C2 simultaneously prefer P2; at most one gets P2; duplicate C1 request returns same token; stale P3 is unavailable; wrong-token release fails. Perform 20 synchronized pairs: zero double allocations, p95 below 250 ms on specified machine.

4.5 S5 — Congestion and obstruction estimation

Owner M3 for monitoring, M5 for simulator inputs. Validate vehicle edge ID against its physical pose corridor, then compute five-second windowed robust speed and per-edge density $\rho_e = \min(1, n_e/K_e)$. Document capacity K_e based on edge length and safe headway. Publish `/traffic/edges` at 2 Hz; classify NORMAL, CONGESTED, BLOCKED or UNKNOWN. Congestion occurs at $\rho_e \geq 0.70$ or speed below 40% of free-flow for three consecutive publications. Obstacle geometry blocks immediately. Never convert stale vehicle observations into a false empty road.

S5 event plan

At start put one vehicle on Cs–J moving at 0.35 m/s. At 20 s add three slow vehicles; at 45 s freeze a source timestamp; at 60 s insert obstacle O across Cs–J. Reject impossible locations and nonmatching edge IDs. Publish transition events with timestamps and chart density, speed, blockage and validity.

Pass: congestion within two seconds of sustained trigger, BLOCKED within one publication, UNKNOWN within 1.2 seconds after loss of all valid observations, and no false congestion in ten free-flow trials. Do not couple this standalone test to motion commands.

4.6 S6 — Optimal graph routing and safe replanning

Owner M4. The map contains the specified undirected vehicle, parking and robot edges; cars alone use parking edges. Construct edge cost

$$c_e = \frac{L_e}{\max(v_e, 0.05)}(1 + 2\rho_e) + w_e,$$

with nonnegative time units. Remove blocked, stale-critical and agent-forbidden edges. Dijkstra or admissible A* computes `ComputeRoute` service response. Replan immediately if the active edge is invalid; otherwise require two-second cooldown and a new path at least 15% cheaper. A route change never moves the agent or transfers crossing ownership.

S6 implementation

Run `code/route_engine.py`, then wrap its `plan()` function as `/route/compute`. Translate current `EdgeStateArray` into edge constraints and weights, validate its snapshot age, and reject requests when critical state is unknown. Replace toy default speeds with validated inputs and test every candidate edge against current conditions before motion.

Pass: baseline path A–Cs–J–E has length 10 m and upper detour A–N–Cn–B–U–E length 14 m. Block Cs–J for detour, also block Cn–B for `NO_ROUTE`. On twenty seeded graphs match independent exhaustive simple-path oracle to 10^{-6} relative cost and report p95 latency below 250 ms.

4.7 S7 — Robot escort with human readiness and cancellation

Owner M4. Implement two separate actions: `/robot/navigate_to` for reusable motion and `/robot/escort` for passenger-assistance policy. Escort state machine: IDLE → TO_PICKUP → WAIT_READY → ESCORT → ARRIVED, with BLOCKED/WAIT_CROSSING pauses and CANCELLED/FAILED/SAFE_STOP outcomes. At R, request readiness through `/robot/confirm_ready` (`std_srvs/Trigger`) and enforce 20-second timeout. Follow graph waypoints with odometry feedback and bounded linear/angular velocity, pausing forward motion during large heading error. Crossing token and obstacle clearance gate motion. On blockage, stop and request a permitted detour; report failure if none exists. Feedback at least 2 Hz. Success requires final distance ≤ 0.20 m sustained one second.

S7 lifecycle contract

Create action interfaces exactly as specified; goal callback validates and atomically reserves the busy flag. Execute via nonblocking state ticks with shared-state protection. Cancel callback authorizes cancellation, and control tick immediately commands zero; terminal reporting waits for verified stop. Clear busy flag on every exception/timeout/terminal route, and never let a late service reply restart a canceled goal.

Pass: ten of ten nominal missions, no crossing or obstacle collision, plus failure tests for no-ready, obstacle, cancel, busy goal, unreachable destination and frozen odometry. Do not market waypoint escort as vision-based person following.

4.8 S8 — Concurrent hub orchestration

Owners M2/M5 with contributions from all. Integrate one bus, one robot and three parking slots. Coordinator prioritizes pedestrians, then uses FIFO for mobile agents after safe clear debounce, preserving a single owner token and occupancy proof. Isolate bus action ID, escort action ID and parking reservation IDs. Avoid duplicate dispatch after late action acknowledgments. Scenario runner injects t=60 s rising demand, t=65 s simultaneous parking requests, t=70 s escort request, t=75 s readiness, t=80–85 s crossing pedestrian, t=90 s blocked road, t=120 s clearance, and t=180–300 s mission completion. Stimuli are scheduled; observed state determines actual timing. **Pass:** ten seeded rush-hour episodes with zero collision, overlapping crossing ownership, double allocation and duplicate bus dispatch. Compare paired reactive/predictive runs using queue wait, crossing wait, travel time and mission success. Publish per-seed results; never disguise safety violations through averaging.

4.9 S9 — Deliberate fault, emergency stop, and recovery

Owner M5, reviewed by everyone. Implement supervisor at 20 Hz using monotonic receipt time and independent adapter final-command expiry. Fault inputs: frozen camera/source stamp, odometry gap over one second, command gap over 0.5 s, planner timeout over one second, duplicate reservation, stale slot sensor, asserted E-stop and disappeared crossing owner. On unknown safety-critical state, inhibit entry. E-stop true latches; E-stop false does *not* reset. `/safety/clear_stop` rejects reset while stop input, hazards, stale sensors or nonzero motion persist. After reset, wait for a *new* mission.

Physical safety

A zero command does not prove stopped wheels. Physical trials require an independent manual power stop and measured braking distance; never test faults by disabling this stop. A barrier must never close on an occupied passage.

Pass: at t=10 s assert stop during robot motion and demonstrate zero output within 0.20 s of supervisor receipt; t=12 s set false and show latch remains; failed reset with stale sensor; successful reset only after fresh clear evidence; no spontaneous restart. Repeat five times at different phases and kill controller/supervisor processes in simulation to verify independent watchdog behavior.

CHAPTER 5

Implementation architecture, concurrency and safety

5.1 Node ownership and communication

Node / member	Build responsibility
scenario_runner / M5	Fixed-step simulation, reset, timed events, private truth; 20 Hz
passenger_detector / M1	Tracking, zones, fresh queue state; 5 Hz
demand_predictor / M1	Episode-trained model and persistence baseline; 1 Hz
bus_manager / M2	DockBus action and door interlocks; control 20 Hz
parking_manager / M3	Occupancy, expiry, atomic reservation services
traffic_monitor / M3	Per-edge state from validated observations; 2 Hz
route_planner / M4	Dijkstra service, graph validity and replanning
robot_manager / M4	NavigateTo/Escort action servers; control 20 Hz
hub_coordinator / M2	One crossing owner, safe queue and dispatch; 10 Hz
safety_supervisor / M5	Candidate gating, watchdog, stop latch; 20 Hz
actuator_bridge / M5	Motor simulation and independent final-command expiry
metrics_logger / M5	Event, model metric and hidden-truth evaluation
dashboard_marker_publisher / M5	Read-only RViz markers and status; 5 Hz

5.2 Crossing ownership sequence

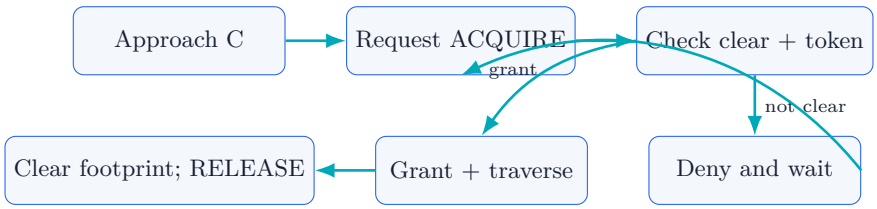


Figure 5.1: Crossing acquisition and release; clearance must be observed even if an owner heartbeat or lease is lost.

5.3 Control-loop pseudocode

1. Read most recent *candidate* command and recorded receipt time.
2. Check stop latch, data age, pose, obstacle sweep, speed cap, crossing ownership and bus door interlock.
3. If any constraint fails, publish a zero final command and a named reason. Otherwise clamp velocity and publish the final command.
4. In the actuator bridge, independently force motor output to zero if final commands stop arriving for 0.5 seconds.
5. Keep safety and actuator code independent of GUI, model-inference service latency and network action cancellation.

The `code/safety_supervisor_example.py` file demonstrates the watchdog pattern only. Its stop latch intentionally starts engaged and *cannot be cleared*: the student must add measured pose, crossing, clearance, door, reset and acknowledgment interfaces before enabling physical control.

5.4 Timer and callback policy

Do not use a blocking mission loop or synchronous service call inside a subscription callback. For long-running actions, choose bounded state-machine timer callbacks, asynchronous service futures, deadlines and carefully chosen callback groups. If an execute callback blocks while odometry and cancellation callbacks must run, select `ReentrantCallbackGroup` and `MultiThreadedExecutor`, then protect shared mission state explicitly. Reservation operations need a lock or serialized callback group. Run motor safety from independent monotonic time.

5.5 TF2 policy and frames

Use `map`, `robot/odom`, `robot/base_link` and bus equivalents; only one transform publisher owns each link. In ideal simulator, map-to-odom may be identity. Camera pixels are not map metres. Never use a camera frame coordinate directly in a robot controller without a calibrated transformation.

CHAPTER 6

Testing, reproducibility, and evidence

6.1 Build–launch–inspect–record cycle

```
source /opt/ros/jazzy/setup.bash
cd ~/smart_hub_ws
rosdep install --from-paths src --ignore-src -r -y
colcon build --symlink-install
source install/setup.bash
ros2 launch smart_hub hub.launch.py scenario:=S8 seed:=42
ros2 node list
ros2 topic list -t
ros2 topic info /hub/queue --verbose
ros2 topic hz /hub/queue
ros2 service list -t
ros2 action list -t
ros2 interface show smart_hub_interfaces/action/EscortPassenger
```

These commands require students to have built their launch file and integrated ROS packages first. They are instructions, not verified terminal output from this deliverable.

```
ros2 bag record -o bags/S8_seed42 \\
 /hub/queue /hub/demand /hub/vehicles /hub/bus_arrival \\
 /parking/state /traffic/edges /crossing/state \\
 /robot/odom /robot/cmd_vel_candidate /robot/cmd_vel \\
 /bus/cmd_vel_candidate /bus/cmd_vel /hub/estop \\
 /hub/safety /hub/events /tf /tf_static
colcon test --packages-select smart_hub
colcon test-result --verbose
```

Add raw camera/input streams and a test-only ground-truth stream when needed. Recording actions' feedback or services requires explicit event instrumentation; ordinary topic listing does not capture all transactions. Do not replay archived velocity commands into physical drivers.

6.2 Mandatory verification matrix

ID	Subsystem	Required verification
T01	S1 counting	Exact stable 0, 3, 5, 4 synthetic sequence
T02	S1 occlusion	No double-count; frozen feed invalid by 1.2 s
T03	S2 prediction	Unseen episodes, MAE/RMSE, persistence comparison
T04	S2 dispatch	One outstanding action; fallback and invalid-forecast protection
T05	S3 docking	Ten nominal dockings, interlock, pose tolerance
T06	S3 cancellation	Canceled action and zero command within 0.5 s
T07	S4 atomicity	Twenty simultaneous request pairs; no duplicate ownership
T08	S4 expiry	Sensor occupied/UNKNOWN cannot be freed; bad token rejected
T09	S5 congestion	Correct NORMAL/CONGESTED/BLOCKED transitions
T10	S5 freshness	Freeze, UNKNOWN, recovery and timestamps
T11	S6 routing	Twenty random edge states versus path oracle
T12	S6 no route	Block Cs–J and Cn–B; explicit NO_ROUTE
T13	S7 escort	Ten nominal successful waypoint missions, 0.20-m tolerance
T14	S7 failures	Busy, readiness timeout, cancel, stale odometry
T15	S8 integration	Ten rush-hour seeds, zero exclusive-resource conflicts
T16	S9 stop latch	Latched stop, rejected/accepted reset, no auto-restart
T17	S9 watchdog	Kill controller then supervisor, observe safe adapter stop
T18	S9 service loss	Planner timeout, bounded retries, no stale-path motion

6.3 Metrics and statistics

$$\mathrm{MAE} = \frac{1}{N}\sum_{i=1}^{N}|y_i - \widehat{y}_i|, \qquad \mathrm{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(y_i - \widehat{y}_i)^2}.$$

$$\mathrm{SuccessRate} = \frac{\text{successful accepted goals}}{\text{accepted goals}}.$$

Report rejected goals separately. Collision means *ground-truth footprint intersection*; crossing conflict means simultaneous incompatible physical occupancy or overlapping owner tokens. A waiting passenger not boarded by run end is censored, not silently dropped. For every run store run ID, scenario, seed, git hash, config/model IDs, hardware details, success and failures. Show individual trials and aggregate statistics with sample counts. Latency between different computers requires synchronized clocks; local durations use monotonic clocks.

6.4 Standalone reference tests actually run during preparation

The pure Python files `route_engine.py`, `parking_engine.py`, and `test_engines.py` were executed with Python unittest in this authoring environment; all five example tests passed. These tests cover baseline path, detour, no-route, request idempotency and unknown-sensor protection. **They do not demonstrate T01–T18 completion, ROS interface compatibility or integrated simulator correctness.**

```
cd code
python3 -m unittest discover -p 'test_*.py' -v
```

CHAPTER 7

Seven-week student plan and graded outputs

Week	Team deliverables	Exit evidence
1	Freeze map, exact IDL schemas, architecture, safety policy and simulator	Build transcript; ROS graph; zero-motion startup
2	S1 perception and S4 parking, unit tests	Queue count sequence and atomic requests
3	S2 persistence + trained forecast; S3 DockBus action	Episode split and action feedback/cancel
4	S5 traffic; S6 graph oracle; S7 escort lifecycle	Valid edge states, routes, robot failures
5	Crossing arbiter, independent safety, S8 and S9	Full pipeline; initial fault log
6	Repeat tests and fix weak cases; optional physical adapter	T01–T18 evidence and bag captures
7	Final reproducibility freeze, technical report and demonstration	Source, metrics, 8-minute video, individual defense

7.1 Team-of-five responsibility map

Member	Primary implementation	Cross-review responsibility
M1	Detector, tracking, demand model, S1/S2 metrics	M3 validity and leakage review
M2	Bus, coordinator, dispatch, crossing, S3/S8	M4 cancellation and ownership review
M3	Parking reservations, traffic monitor, S4/S5	M1 labels and atomicity review
M4	Graph, Dijkstra, robot actions, S6/S7	M2 crossing arbitration review
M5	Simulator, safety, actuator bridge, launch, logs, S9	Integrated fault and stop proof

The source lists traffic monitoring under both M3 and M5 in different sections. This guide treats M3 as algorithm owner and M5 as simulator-input/integration owner; the instructor should confirm that handoff without changing either graded scenario. If there are up to five class teams, *each* team builds the full baseline. Team specialization applies to optional extensions only.

7.2 Assessment — source rubric unchanged

Area	Points
ROS 2 architecture	10
Scenario implementation	14
Testing quality	10
Safety behavior	10
AI prediction	8
Routing and coordination	10

Area	Points
Code quality	6
Documentation	6
Demonstration performance	6
Team subtotal	80
Individual contribution	20
Total	**100**

7.3 Eight-minute demonstration storyboard

One minute map/architecture; two minutes queue, demand dispatch, bus, parking; two minutes escort and replan; two minutes fault/cancellation/reset; one minute metrics and limitations. Submit all source, configuration, scenario YAMLs, models, training/evaluation scripts, safe video evidence and a suggested 12–18-page technical report. Every individual explains one topic path, one service/action lifecycle, and one repaired failure.

CHAPTER 8

Prioritized optional engineering improvements

Grading boundary — The table below is optional. No improvement replaces the mandated simulation-first eight functional scenarios or S9 safety evidence. Implement enhancements only after the integrated baseline passes.

Optional improvement	Demonstration value	Difficulty	Effort	Preconditions / risk
Structured event dashboard	Explain every dispatch and denial in real time	Low	4–8 h	Read-only access
Automatic trace checker	Turns scenario YAML into pass/-fail event assertions	Medium	8–14 h	Good event IDs
Parameter sweep script	Calibrate threshold and timeout tradeoffs	Low	5–9 h	Reproducible seeds
Better queue tracking	Improve partial occlusion robustness	Medium	10–20 h	Labeled video and consent
Prediction uncertainty	Show conservative dispatch when model unsure	Medium	10–18 h	Calibrated held-out data
Multi-agent collision predictor	Predict swept footprint intersections before crossing	High	16–28 h	Strong baseline safety suite
Gazebo/RViz integration	More realistic visualization, sensor physics	High	20–40 h	Compatible Jazzy/Gazebo version
Nav2 localization/navigation	Compare graph waypoint control with Nav2	High	24–45 h	Valid maps, TF and odom
Optional tablet/voice aid	Accessible assistive feedback without identity collection	Medium	12–24 h	Human-in-loop failover
Tabletop physical replication	Confirm timing, power and real stopping distance	High	25–50 h	Hardwired E-stop + risk review

Recommended order: structured logging, automated trace checker and parameter sweep offer high verification value at low cost. Implement more complex perception only with lawful data, and physical hardware only after S9 tests are demonstrated.

8.1 Practical extension design exercise

Choose one improvement; write its measurable hypothesis, changed files, extra dependencies, cost, risk, expected metric, negative test and rollback plan. Compare the extension to an unchanged baseline on the same seeds. Do not report performance gain without matching evaluation conditions.

CHAPTER 9

Coverage checklist and quality-verification report

9.1 Requirement-to-implementation traceability

Source	Mandatory requirement	Guide implementation / evidence	Owner
§2–3	Map M1 and simulator, swept collision	Chapters 3, 5; S9; scenario trace	M5
§4–7	Nodes, typed topics, services, actions, data validity	Chapter 3 and scenario contracts; IDL to be built	All
§8	Queue counting and video demo	S1 + T01/T02	M1
§9	Held-out regression and persistence	S2 + T03/T04	M1/M2
§10	DockBus, interlocks, cancel	S3 + T05/T06	M2
§11	Slot freshness and reservation atomicity	S4 + T07/T08	M3
§12	Edge congestion, obstruction, stale flags	S5 + T09/T10	M3/M5
§13	Cost-optimal routing, no-route	S6 + T11/T12	M4
§14	Escort/Navigation and readiness	S7 + T13/T14	M4
§15	Concurrent timed scenario	S8 + T15	All
§16	Fault handling and stop latch	S9 + T16–T18	M5
§17–20	Communication matrix, algorithms, repository, bags	Chapters 2–6 and code appendix	All
§21	Metrics and reproducible tests	Chapter 6 test matrix	All
§22	Optional tabletop implementation	Chapters 1, 5, 8	All
§23–24	Weekly plan, grading, deliverables	Chapter 7	All

9.2 Submission readiness checks

- ☐ All original custom `.msg`, `.srv` and `.action` files match the assignment's field lists.
- ☐ One clean build and parameter/version snapshot is included.
- ☐ Each scenario S1–S9 has input YAML, sample output, negative test and measurable evidence.
- ☐ T01–T18 includes trial count, pass/fail, observed value and artifact path.
- ☐ Exclusive crossing and final-command ownership hold under failure and cancellation.
- ☐ Code, data, model artifact, tests, logs and license notices are archived.
- ☐ Every teammate has code commits, test evidence and cross-review notes.

9.3 Verification performed for this guide

Check	Status
Assignment source studied	All 35 pages available; sections 1–24 reflected
Scenario coverage	S1–S9 explicitly documented
Test coverage in text	T01–T18 mapped
Python isolated logic	Five local `unittest` tests passed
ROS package build / execution	**Not verified:** ROS 2 Jazzy not installed here

Check	Status
Complete ROS implementation	**Not delivered:** student implementation remains required
Hardware	**Not tested**; simulation-first handbook only
LaTeX compilation	See accompanying QA report generated after compile

Scope of the companion code

The `code/` materials are reference exercises, not a full replacement for the assignment's assessed repository. For example, the parking example omits the required sensor debounce and ROS service wrappers, and the simple safety example lacks clearance and reset handling. These are intentionally identified as student implementation milestones.

CHAPTER A

Companion code: pure Python route planner

Listing A.1: Route engine (demonstration module)

```
"""Pure-Python graph planner for M1 (educational reference)."""
import heapq
import math

NODES = {'A':(1,2),'Cs':(5,2),'J':(9,2),'E':(11,2),'N':(1,4),
         'Cn':(5,4),'B':(8,4),'U':(11,4),'R':(3,6),'W':(5,6),
         'D':(9,1),'P1':(8,6.5),'P2':(9.5,6.5),'P3':(11,6.5)}
VEHICLE = [('A','Cs'),('Cs','J'),('J','E'),('A','N'),('N','Cn'),
           ('Cn','B'),('B','U'),('U','E')]
PARK = [('U','P3'),('P3','P2'),('P2','P1')]
ROBOT = [('R','W'),('W','Cn'),('Cn','Cs'),('Cs','J'),('J','D'),
         ('Cn','B'),('B','U'),('U','E'),('E','J')]

def edge_id(a,b):
    return '-'.join(sorted((a,b)))

def allowed_edges(agent):
    if agent == 'robot': return ROBOT
    if agent == 'bus': return VEHICLE
    if agent == 'car': return VEHICLE + PARK
    raise ValueError('Unsupported agent type')

def plan(start,goal,agent='car',blocked=(),density=None,speed=None,wait=None):
    """Dijkstra. Invalid edge states should be added to blocked by caller."""
    if start not in NODES or goal not in NODES:
        raise ValueError('Unknown node')
    density, speed, wait = density or {}, speed or {}, wait or {}
    graph={node:[] for node in NODES}
    for a,b in allowed_edges(agent):
        e=edge_id(a,b)
        if e in blocked: continue
        rho=density.get(e,0.0); v=speed.get(e,0.35); w=wait.get(e,0.0)
        if not (0 <= rho <= 1 and math.isfinite(v) and v > 0 and w >= 0):
            continue
        length=math.dist(NODES[a], NODES[b]); cost=length/max(v,0.05)*(1+2*rho)+w
        graph[a].append((b,cost)); graph[b].append((a,cost))
    q=[(0.0,start,(start,))]; best={start:0.0}
    while q:
        cost,u,path=heapq.heappop(q)
        if cost>best[u]+1e-10: continue
        if u==goal: return list(path),cost
        for v,w in graph[u]:
            alt=cost+w
            if alt<best.get(v,float('inf')):
                best[v]=alt; heapq.heappush(q,(alt,v,path+(v,)))
    return [],float('inf')

if __name__=='__main__':
    for blocked in (set(),{edge_id('Cs','J')},
                    {edge_id('Cs','J'),edge_id('Cn','B')}):
        print(sorted(blocked),plan('A','E','car',blocked))
```

CHAPTER B

Companion code: parking domain logic

Listing B.1: Reservation algorithm (debouncing and ROS wrappers remain student tasks)

```
"""Thread-safe reservation domain logic; sensor freshness supplied by caller."""
from dataclasses import dataclass
from threading import RLock
from uuid import uuid4
import time

@dataclass
class Slot:
    observed: str = 'UNKNOWN'  # FREE/OCCUPIED/UNKNOWN
    observed_at: float = float('-inf')
    reservation: str = ''
    vehicle: str = ''
    expiry: float = 0.0

class ParkingEngine:
    def __init__(self,slots=('P1','P2','P3'),freshness=1.0):
        self.slots={k:Slot() for k in slots}; self.freshness=freshness
        self.cache={}; self.lock=RLock()

    def observe(self,slot,occupied,valid=True,now=None):
        now=time.monotonic() if now is None else now
        with self.lock:
            s=self.slots[slot]
            s.observed='OCCUPIED' if occupied else 'FREE' if valid else 'UNKNOWN'
            s.observed_at=now
            # Production node must debounce readings for 0.5 seconds.

    def state(self,s,now):
        if now-s.observed_at>self.freshness or s.observed=='UNKNOWN': return 'UNKNOWN'
        if s.observed=='OCCUPIED': return 'OCCUPIED'
        if s.reservation and s.expiry>now: return 'RESERVED'
        return 'FREE'

    def reserve(self,request_id,vehicle,preferred='',ttl=30,now=None):
        now=time.monotonic() if now is None else now
        payload=(vehicle,preferred,ttl)
        with self.lock:
            if request_id in self.cache:
                old_payload,result=self.cache[request_id]
                return result if old_payload==payload else (False,'','', 'KEY_REUSE')
            if not request_id or not vehicle or not 5<=ttl<=120:
                return (False,'','','INVALID_REQUEST')
            self.expire(now)
            if any(s.vehicle==vehicle and s.reservation for s in self.slots.values()):
                result=(False,'','','VEHICLE_ALREADY_RESERVED')
            else:
                options=[k for k,s in self.slots.items() if self.state(s,now)=='FREE']
                if preferred and preferred in options: chosen=preferred
                elif preferred and preferred not in options: chosen=''
                else: chosen=options[0] if options else ''
                if not chosen: result=(False,'','','NO_SLOT')
                else:
                    token=uuid4().hex; s=self.slots[chosen]
                    s.reservation=token; s.vehicle=vehicle; s.expiry=now+ttl
                    result=(True,token,chosen,'OK')
            self.cache[request_id]=(payload,result)
            return result

    def expire(self,now=None):
        now=time.monotonic() if now is None else now
        with self.lock:
            for s in self.slots.values():
                if s.reservation and s.expiry<=now:
                    s.reservation=''; s.vehicle=''; s.expiry=0
```

```
    def release(self,token,vehicle):
        with self.lock:
            for s in self.slots.values():
                if s.reservation==token and s.vehicle==vehicle:
                    s.reservation=''; s.vehicle=''; s.expiry=0
                    return True
            return False
```

CHAPTER C

Companion code: local unit tests

Listing C.1: Five passing pure-Python unit tests

```
import unittest
from route_engine import plan,edge_id
from parking_engine import ParkingEngine

class TestRouting(unittest.TestCase):
    def test_baseline(self):
        path,_=plan('A','E')
        self.assertEqual(path,['A','Cs','J','E'])
    def test_detour(self):
        path,_=plan('A','E',blocked={edge_id('Cs','J')})
        self.assertEqual(path,['A','N','Cn','B','U','E'])
    def test_no_route(self):
        path,_=plan('A','E',blocked={edge_id('Cs','J'),edge_id('Cn','B')})
        self.assertEqual(path,[])

class TestParking(unittest.TestCase):
    def test_idempotent_and_exclusive(self):
        e=ParkingEngine(); e.observe('P2',False,now=10);e.observe('P3',False,now=10)
        a=e.reserve('a','C1','P2',now=10)
        self.assertEqual(a,e.reserve('a','C1','P2',now=10))
        b=e.reserve('b','C2','P2',now=10)
        self.assertFalse(b[0])
        self.assertTrue(a[0])
    def test_unknown_never_free(self):
        e=ParkingEngine();e.observe('P1',False,now=10)
        self.assertFalse(e.reserve('r','car',now=12)[0])

if __name__=='__main__': unittest.main()
```

CHAPTER D

Companion ROS 2 teaching examples

Listing D.1: Synthetic QueueState publisher example

```
"""Runnable once smart_hub_interfaces is built; deterministic teaching input."""
import rclpy
from rclpy.node import Node
from smart_hub_interfaces.msg import QueueState

class QueueDemo(Node):
    def __init__(self):
        super().__init__('queue_demo')
        self.pub=self.create_publisher(QueueState,'/hub/queue',10)
        self.start=self.get_clock().now()
        self.create_timer(0.2,self.tick)

    def tick(self):
        t=(self.get_clock().now()-self.start).nanoseconds/1e9
        msg=QueueState(); msg.header.stamp=self.get_clock().now().to_msg()
        msg.header.frame_id='map';msg.zone_id='Q'
        msg.count=0 if t<5 else 3 if t<10 else 5 if t<15 else 4
        msg.confidence=1.0;msg.valid=t<30
        msg.crossing_occupied=20<=t<24
        self.pub.publish(msg)

def main(args=None):
    rclpy.init(args=args);node=QueueDemo()
    try: rclpy.spin(node)
    finally: node.destroy_node();rclpy.shutdown()

if __name__=='__main__': main()
```

Listing D.2: Incomplete teaching-only watchdog pattern

```
"""Teaching-only command watchdog, not a certified safety controller."""
import time
import rclpy
from rclpy.node import Node
from geometry_msgs.msg import Twist, TwistStamped
from std_msgs.msg import Bool

class SafetyDemo(Node):
    def __init__(self):
        super().__init__('safety_demo')
        self.last=None;self.cmd=None;self.latched=True
        self.create_subscription(TwistStamped,'/robot/cmd_vel_candidate',self.on_cmd,1)
        self.create_subscription(Bool,'/hub/estop',self.on_estop,1)
        self.pub=self.create_publisher(Twist,'/robot/cmd_vel',1)
        self.create_timer(0.05,self.tick)

    def on_cmd(self,msg):
        self.cmd=msg.twist;self.last=time.monotonic()

    def on_estop(self,msg):
        if msg.data: self.latched=True
        # False never clears the latch; reset service omitted in demo.

    def tick(self):
        out=Twist()
        if not self.latched and self.last is not None and time.monotonic()-self.last<=0.5:
            out.linear.x=max(-0.25,min(0.25,self.cmd.linear.x))
            out.angular.z=max(-0.8,min(0.8,self.cmd.angular.z))
        self.pub.publish(out)

def main(args=None):
    rclpy.init(args=args);n=SafetyDemo()
    try: rclpy.spin(n)
    finally:n.destroy_node();rclpy.shutdown()
```

```
if __name__==’__main__’:main()
```